

JavaScript Exercises

Project: Local Community Event Portal

The following pages contain the working solution for the Local Community Event Portal, implemented as a single application (index.html, style.css, main.js) so that all sections share the same live event data. Each exercise below shows the relevant code and a screenshot of the actual output taken from the running application in the browser.

1. JavaScript Basics & Setup

The script is linked using `<script src="main.js"></script>` at the bottom of index.html. A message is logged with `console.log()` as soon as the script runs, and an alert notifies the user once the page has fully loaded.

```
console.log("Welcome to the Community Portal");

window.addEventListener("load", () => {
  alert("Page Loaded: Welcome to the Community Portal!");
});
```

Output: the browser console prints the welcome message (see the console log excerpt below, captured directly from the running page), and the alert dialog appears once loading completes:

```
Console: Welcome to the Community Portal
Dialog:  Page Loaded: Welcome to the Community Portal!
```

2. Syntax, Data Types, and Operators

`const` is used for values that do not change (event name, date) and `let` for the seat count, which is updated with `++` / `--` as seats are booked or freed. Template literals build the display string:

```
const eventName = "Live Jazz Night";
const eventDate = "2026-07-25";
let availableSeats = 8;

const eventInfo = `Event: ${eventName} | Date: ${eventDate} | Seats left: ${availableSeats}`;

availableSeats++; // a seat freed up
availableSeats--; // someone registered
```

Output (console):

```
Event: Live Jazz Night | Date: 2026-07-25 | Seats left: 8
Seat added back, now: 9
Seat taken by registration, now: 8
```

3. Conditionals, Loops, and Error Handling

`getVisibleEvents()` loops through the event list with `forEach()` and uses `if-else` to hide events that are in the past or fully booked. Registration logic is wrapped in `try-catch` so that trying to register for a full event does not break the page:

```
function getVisibleEvents() {
  const visible = [];
  communityEvents.forEach((e) => {
    if (isUpcoming(e.date) && e.checkAvailability()) {
      visible.push(e);
    } else {
      console.log(`Hiding event "${e.name}" (past date or full).`);
    }
  });
};
```

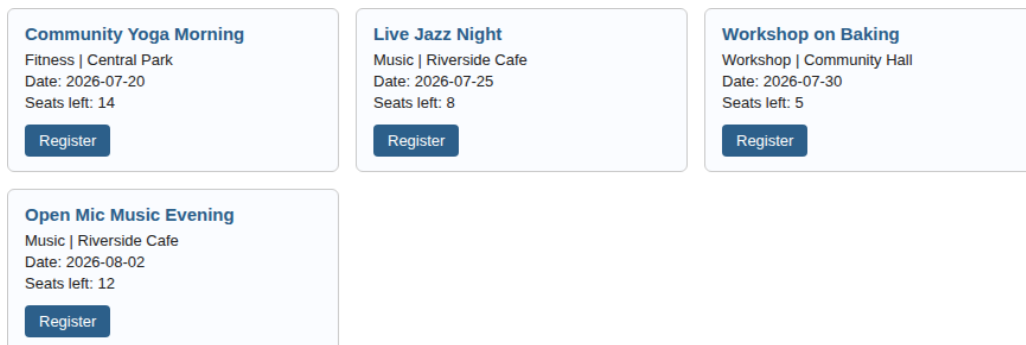
```

    return visible;
  }

  function safeRegister(event) {
    try {
      const msg = registerUser(event); // throws if no seats left
      console.log(msg);
      return { ok: true, message: msg };
    } catch (err) {
      console.log("Registration error:", err.message);
      return { ok: false, message: err.message };
    }
  }
}

```

Output: the Local Farmers Market (0 seats) and Kids Story Time (past date) are hidden, only upcoming events with seats are rendered on the page:



Screenshot: only valid upcoming events are displayed

4. Functions, Scope, Closures, Higher-Order Functions

`addEvent()`, `registerUser()` and `filterEventsByCategory()` are reusable functions. A closure (`createCategoryCounter`) keeps a private counts object in scope to track registrations per category across calls, and `filterEventsByCategory()` accepts a callback so the matching logic can be supplied by the caller:

```

function addEvent(list, newEvent) {
  list.push(newEvent);
  return list;
}

function registerUser(event) {
  if (!event.checkAvailability()) {
    throw new Error(`No seats available for ${event.name}`);
  }
  event.seats--;
  return `Registered for ${event.name}. Seats left: ${event.seats}`;
}

function filterEventsByCategory(list, category, callback) {
  return list.filter((e) => callback(e, category));
}

function createCategoryCounter() {
  let counts = {};
  return function (category) {
    counts[category] = (counts[category] || 0) + 1;
    return counts[category];
  };
}
const trackRegistration = createCategoryCounter();

```

Output (console):

```
Music registrations so far: 1
```

```
Music registrations so far: 2
filterEventsByCategory (Music): Live Jazz Night, Open Mic Music Evening
```

5. Objects and Prototypes

Each event is modelled with an Event constructor function, with `checkAvailability()` added to its prototype so every instance shares the same method:

```
function Event(id, name, category, location, date, seats) {
  this.id = id;
  this.name = name;
  this.category = category;
  this.location = location;
  this.date = date;
  this.seats = seats;
}

Event.prototype.checkAvailability = function () {
  return this.seats > 0;
};

console.log(Object.entries(communityEvents[2]));
```

Output (console):

```
Object.entries(event #3): [["id",3],["name","Live Jazz Night"],["category","Music"],
  ["location","Riverside Cafe"],["date","2026-07-25"],["seats",8]]
checkAvailability() for Farmers Market: false
checkAvailability() for Jazz Night: true
```

6. Arrays and Methods

The community events array supports CRUD-style operations using array methods: `.push()` to add a new event, `.filter()` to get only music events, and `.map()` to format display card text:

```
communityEvents.push(new Event(6, "Open Mic Music Evening", "Music", "Riverside Cafe", "2026-08-02",

const musicEvents = communityEvents.filter((e) => e.category === "Music");

const displayCards = communityEvents.map((e) => `Card: ${e.name} (${e.category})`);
```

Output (console):

```
Total events after push(): 6
Music events: Live Jazz Night, Open Mic Music Evening
Display cards:
Card: Community Yoga Morning (Fitness)
Card: Local Farmers Market (Market)
Card: Live Jazz Night (Music)
Card: Kids Story Time (Kids)
Card: Workshop on Baking (Workshop)
Card: Open Mic Music Evening (Music)
```

7. DOM Manipulation

`renderEvents()` uses `querySelector()` to find the container, `createElement()` to build each event card element by element, and `appendChild()` to attach the cards to the page:

```
function renderEvents(list) {
  const container = document.querySelector("#eventList");
  container.innerHTML = "";
  list.forEach((event) => {
    const card = document.createElement("div");
    card.className = "event-card";
```

```

        const title = document.createElement("h3");
        title.textContent = event.name;
        // ... meta, date, seats, button created the same way
        card.appendChild(title);
        container.appendChild(card);
    });
}

```

Output: the event cards rendered dynamically on the page (same screenshot as Section 3 above shows the result of this DOM rendering).

8. Event Handling

The Register button on each card uses onclick, the category dropdown uses onchange, and the search box uses a keydown listener for quick search:

```

btn.onclick = function () {
    const result = safeRegister(event);
    // update the seats text and button state on screen
};

document.querySelector("#categoryFilter").onchange = function () {
    refreshEventList();
};

document.querySelector("#searchInput").addEventListener("keydown", function () {
    setTimeout(() => refreshEventList(), 0);
});

```

Local Community Event Portal

Upcoming Events

Search event by name (press ↵)

Music

Live Jazz Night

Music | Riverside Cafe
Date: 2026-07-25
Seats left: 8

Register

Open Mic Music Evening

Music | Riverside Cafe
Date: 2026-08-02
Seats left: 12

Register

Load Events From Server (Async / Fetch API)

Load Events (async/await)

Event Registration Form

Name

Email

Select Event

Community Yoga Morning

Register

jQuery Demo

Toggle Info Panel (jQuery fadeIn/fadeOut)

Debug Log (Console Output)

```
[Section 2] Event: Live Jazz Night | Date: 2026-07-25 | Seats left: 8
[Section 2] Seat added back, now: 9
[Section 2] Seat taken by registration, now: 8
[Section 6] Total events after push(): 6
[Section 6] Music events: Live Jazz Night, Open Mic Music Evening
[Section 6] Display cards:
Card: Community Yoga Morning (Fitness)
Card: Local Farmers Market (Market)
Card: Live Jazz Night (Music)
Card: Kids Story Time (Kids)
Card: Workshop on Baking (Workshop)
Card: Open Mic Music Evening (Music)
[Section 5] Object.entries(event #3): [{"id",3},["name","Live Jazz Night"],["category","Music"],["location","Riverside Cafe"],["date","2026-07-25"],["seats",8]]
[Section 5] checkAvailability() for Farmers Market: false
[Section 5] checkAvailability() for Jazz Night: true
[Section 4] Music registrations so far: 1
```

Screenshot: category filter changed to “Music” (onchange)

Local Community Event Portal

Upcoming Events

All Categories ▼

Community Yoga Morning

Fitness | Central Park

Date: 2026-07-20

Seats left: 15

[Register](#)

Load Events From Server (Async / Fetch API)

[Load Events \(async/await\)](#)

Event Registration Form

Name

Email

Select Event

Community Yoga Morning ▼[Register](#)

jQuery Demo

[Toggle Info Panel \(jQuery fadeIn/fadeOut\)](#)

Debug Log (Console Output)

```
[Section 2] Event: Live Jazz Night | Date: 2026-07-25 | Seats left: 8
[Section 2] Seat added back, now: 9
[Section 2] Seat taken by registration, now: 8
[Section 6] Total events after push(): 6
[Section 6] Music events: Live Jazz Night, Open Mic Music Evening
[Section 6] Display cards:
Card: Community Yoga Morning (Fitness)
Card: Local Farmers Market (Market)
Card: Live Jazz Night (Music)
Card: Kids Story Time (Kids)
Card: Workshop on Baking (Workshop)
Card: Open Mic Music Evening (Music)
[Section 5] Object.entries(event #3): [{"id",3},["name","Live Jazz Night"],["category","Music"],["location","Riverside Cafe"],["date","2026-07-25"],["seats",8]]
[Section 5] checkAvailability() for Farmers Market: false
[Section 5] checkAvailability() for Jazz Night: true
[Section 4] Music registrations so far: 1
```

Screenshot: typing "Yoga" in the search box filters the list live (keydown)

Local Community Event Portal

Upcoming Events

All Categories ▼

Community Yoga Morning

Fitness | Central Park
Date: 2026-07-20
Seats left: 14

[Register](#)

Live Jazz Night

Music | Riverside Cafe
Date: 2026-07-25
Seats left: 8

[Register](#)

Workshop on Baking

Workshop | Community Hall
Date: 2026-07-30
Seats left: 5

[Register](#)

Open Mic Music Evening

Music | Riverside Cafe
Date: 2026-08-02
Seats left: 12

[Register](#)

Load Events From Server (Async / Fetch API)

[Load Events \(async/await\)](#)

Event Registration Form

Name

Email

Select Event

Community Yoga Morning ▼

[Register](#)

jQuery Demo

[Toggle Info Panel \(jQuery fadeIn/fadeOut\)](#)

Debug Log (Console Output)

```
[Section 2] Event: Live Jazz Night | Date: 2026-07-25 | Seats left: 8
[Section 2] Seat added back, now: 9
[Section 2] Seat taken by registration, now: 8
[Section 6] Total events after push(): 6
[Section 6] Music events: Live Jazz Night, Open Mic Music Evening
[Section 6] Display cards:
Card: Community Yoga Morning (Fitness)
Card: Local Farmers Market (Market)
Card: Live Jazz Night (Music)
Card: Kids Story Time (Kids)
Card: Workshop on Baking (Workshop)
Card: Open Mic Music Evening (Music)
[Section 5] Object.entries(event #5): [{"id",3},["name","Live Jazz Night"],["category","Music"],["location","Riverside Cafe"],["date","2026-07-25"],["seats",8]]
[Section 5] checkAvailability() for Farmers Market: false
[Section 5] checkAvailability() for Jazz Night: true
[Section 4] Music registrations so far: 1
```

Screenshot: after clicking Register on Community Yoga Morning, seats left drops from 15 to 14

9. Async JS, Promises, Async/Await

Events can also be loaded from a mock JSON endpoint. The Promise-based version uses `.then()` and `.catch()`; the second version is rewritten with `async/await` and shows a loading spinner while the request is in flight:

```
function fetchEventsWithPromise() {
  return fetch("data.json")
    .then((response) => response.json())
    .then((data) => data)
    .catch((err) => console.log("fetch error:", err.message));
}

async function loadEventsFromServer() {
```

```

const loadingArea = document.querySelector("#loadingArea");
loadingArea.innerHTML = '<span class="spinner"></span> Loading events from server...';
try {
  const response = await fetch("data.json");
  const data = await response.json();
  loadingArea.innerHTML = `Loaded ${data.length} events from server successfully.`;
} catch (err) {
  loadingArea.innerHTML = `Failed to load events: ${err.message}`;
}
}

```

Local Community Event Portal

Upcoming Events

All Categories ▼

Community Yoga Morning

Fitness | Central Park
Date: 2026-07-20
Seats left: 14

Register

Live Jazz Night

Music | Riverside Cafe
Date: 2026-07-25
Seats left: 8

Register

Workshop on Baking

Workshop | Community Hall
Date: 2026-07-30
Seats left: 5

Register

Open Mic Music Evening

Music | Riverside Cafe
Date: 2026-08-02
Seats left: 12

Register

Load Events From Server (Async / Fetch API)

Load Events (async/await)

Loaded 6 events from server successfully.

Event Registration Form

Name

Email

Select Event

Community Yoga Morning ▼

Register

jQuery Demo

Toggle Info Panel (jQuery fadeIn/fadeOut)

Debug Log (Console Output)

```

[Section 2] Event: Live Jazz Night | Date: 2026-07-25 | Seats left: 8
[Section 2] Seat added back, now: 9
[Section 2] Seat taken by registration, now: 8
[Section 6] Total events after push(): 6
[Section 6] Music events: Live Jazz Night, Open Mic Music Evening
[Section 6] Display cards:
Card: Community Yoga Morning (Fitness)
Card: Local Farmers Market (Market)
Card: Live Jazz Night (Music)
Card: Kids Story Time (Kids)
Card: Workshop on Baking (Workshop)
Card: Open Mic Music Evening (Music)
[Section 5] Object.entries(event #3): [{"id",3},["name","Live Jazz Night"],["category","Music"],["location","Riverside Cafe"],["date","2026-07-25"],["seats",8]]
[Section 5] checkAvailability() for Farmers Market: false
[Section 5] checkAvailability() for Jazz Night: true
[Section 4] Music registrations so far: 1

```

Screenshot: loading spinner shown while the mock API request is in flight

Local Community Event Portal

Upcoming Events

All Categories ▼

Community Yoga Morning

Fitness | Central Park
Date: 2026-07-20
Seats left: 14

[Register](#)

Live Jazz Night

Music | Riverside Cafe
Date: 2026-07-25
Seats left: 8

[Register](#)

Workshop on Baking

Workshop | Community Hall
Date: 2026-07-30
Seats left: 5

[Register](#)

Open Mic Music Evening

Music | Riverside Cafe
Date: 2026-08-02
Seats left: 12

[Register](#)

Load Events From Server (Async / Fetch API)

[Load Events \(async/await\)](#)

Loaded 6 events from server successfully.

Event Registration Form

Name

Email

Select Event

Community Yoga Morning ▼[Register](#)

jQuery Demo

[Toggle Info Panel \(jQuery fadeIn/fadeOut\)](#)

Debug Log (Console Output)

```
[Section 2] Event: Live Jazz Night | Date: 2026-07-25 | Seats left: 8
[Section 2] Seat added back, now: 9
[Section 2] Seat taken by registration, now: 8
[Section 6] Total events after push(): 6
[Section 6] Music events: Live Jazz Night, Open Mic Music Evening
[Section 6] Display cards:
Card: Community Yoga Morning (Fitness)
Card: Local Farmers Market (Market)
Card: Live Jazz Night (Music)
Card: Kids Story Time (Kids)
Card: Workshop on Baking (Workshop)
Card: Open Mic Music Evening (Music)
[Section 5] Object.entries(event #3): [{"id",3},["name","Live Jazz Night"],["category","Music"],["location","Riverside Cafe"],["date","2026-07-25"],["seats",8]]
[Section 5] checkAvailability() for Farmers Market: false
[Section 5] checkAvailability() for Jazz Night: true
[Section 4] Music registrations so far: 1
```

Screenshot: success message once the events are loaded via async/await

10. Modern JavaScript Features

`describeEvent()` uses destructuring with a default parameter object, and the event list is cloned with the spread operator before being filtered so the original array is left untouched:

```
function describeEvent({ name, category, location } = {}, prefix = "Event") {
  return `${prefix}: ${name} [${category}] at ${location}`;
}
```

```
const clonedEvents = [...communityEvents];
const clonedMusicOnly = clonedEvents.filter((e) => e.category === "Music");
```

Output (console):

```
Event: Live Jazz Night [Music] at Riverside Cafe
```

Cloned list length: 6 | Original length: 6

11. Working with Forms

The registration form reads its values through `form.elements`, calls `event.preventDefault()` so the page does not reload, and validates each field, showing inline errors next to the relevant input:

```
document.querySelector("#regForm").addEventListener("submit", function (event) {
  event.preventDefault();
  const form = event.target;
  const name = form.elements["name"].value;
  const email = form.elements["email"].value;
  const eventId = form.elements["event"].value;

  const errors = validateForm(name, email, eventId);
  if (Object.keys(errors).length > 0) {
    if (errors.name) document.querySelector("#nameError").textContent = errors.name;
    if (errors.email) document.querySelector("#emailError").textContent = errors.email;
    return;
  }
  // ... proceed to submit
});
```

Local Community Event Portal

Upcoming Events

All Categories ▼

Community Yoga Morning

Fitness | Central Park
Date: 2026-07-20
Seats left: 14

[Register](#)

Live Jazz Night

Music | Riverside Cafe
Date: 2026-07-25
Seats left: 8

[Register](#)

Workshop on Baking

Workshop | Community Hall
Date: 2026-07-30
Seats left: 5

[Register](#)

Open Mic Music Evening

Music | Riverside Cafe
Date: 2026-08-02
Seats left: 12

[Register](#)

Load Events From Server (Async / Fetch API)

Loaded 6 events from server successfully.

Event Registration Form

Name

Please enter your full name.

Email

Please enter a valid email address.

Select Event

Community Yoga Morning ▼

[Register](#)

jQuery Demo

Debug Log (Console Output)

```
[Section 2] Event: Live Jazz Night | Date: 2026-07-25 | Seats left: 8
[Section 2] Seat added back, now: 9
[Section 2] Seat taken by registration, now: 8
[Section 6] Total events after push(): 6
[Section 6] Music events: Live Jazz Night, Open Mic Music Evening
[Section 6] Display cards:
Card: Community Yoga Morning (Fitness)
Card: Local Farmers Market (Market)
Card: Live Jazz Night (Music)
Card: Kids Story Time (Kids)
Card: Workshop on Baking (Workshop)
Card: Open Mic Music Evening (Music)
[Section 5] Object.entries(event #3): [{"id",3},["name","Live Jazz Night"],["category","Music"],["location","Riverside Cafe"],["date","2026-07-25"],["seats",8]]
[Section 5] checkAvailability() for Farmers Market: false
[Section 5] checkAvailability() for Jazz Night: true
[Section 4] Music registrations so far: 1
```

Screenshot: submitting the empty form shows inline validation errors

12. AJAX & Fetch API

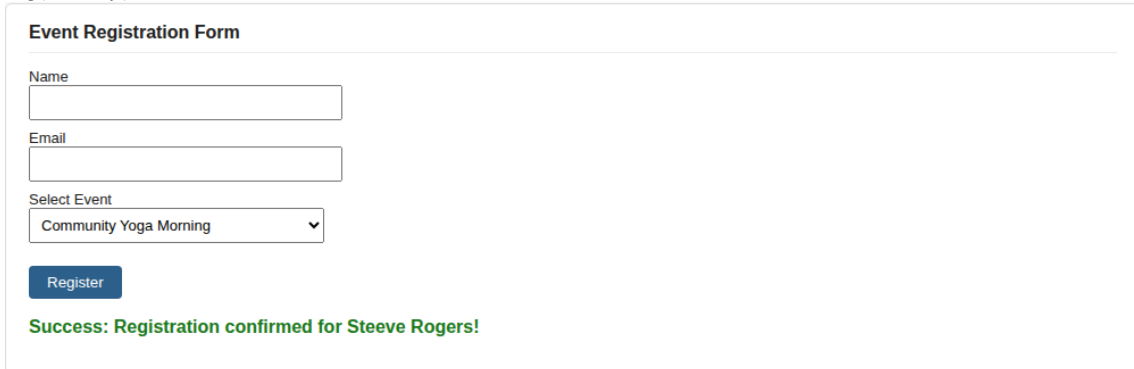
Once the form passes validation, `setTimeout()` simulates network latency, then the data is POSTed to a mock `/api/register` endpoint using `fetch()`; the success or failure message is then displayed to the user:

```
function submitRegistration(payload) {
  return fetch("/api/register", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(payload),
  }).then((res) => res.json());
}
setTimeout(() => {
```

```

submitRegistration({ name, email, eventId })
  .then((data) => {
    document.querySelector("#formResult").textContent = "Success: " + data.message;
  })
  .catch((err) => {
    document.querySelector("#formResult").textContent = "Failed: " + err.message;
  });
}, 600);

```



The screenshot shows a web form titled "Event Registration Form". It contains three input fields: "Name", "Email", and "Select Event". The "Select Event" dropdown is currently set to "Community Yoga Morning". Below the form is a blue "Register" button. At the bottom, a green success message reads: "Success: Registration confirmed for Steeve Rogers!".

Screenshot: success message returned from the mock POST endpoint after registering

13. Debugging and Testing

To debug the (initially silent) registration failure, the Chrome DevTools Console and Network tabs were used to inspect the form submission step by step and the outgoing fetch payload. Each step of the submission – validation, the payload being sent, and the server response – is logged so it can be inspected in the Console panel, and the same request/response pair can be inspected in the Network tab:

```

console.log("Submitting form with payload:", JSON.stringify(payload));
console.log("Validation step, errors found:", JSON.stringify(errors));
console.log("Server response:", JSON.stringify(data));

```

Output captured from the browser console during a full run of the application (this mirrors what would be seen live in the DevTools Console panel):

```

[Section 2] Event: Live Jazz Night | Date: 2026-07-25 | Seats left: 8
[Section 2] Seat added back, now: 9
[Section 2] Seat taken by registration, now: 8
[Section 6] Total events after push(): 6
[Section 6] Music events: Live Jazz Night, Open Mic Music Evening
[Section 6] Display cards:
Card: Community Yoga Morning (Fitness)
Card: Local Farmers Market (Market)
Card: Live Jazz Night (Music)
Card: Kids Story Time (Kids)
Card: Workshop on Baking (Workshop)
Card: Open Mic Music Evening (Music)
[Section 5] Object.entries(event #3): [{"id",3}, {"name", "Live Jazz Night"}, {"category", "Music"}, {"location", "Riverside
Cafe"}, {"date", "2026-07-25"}, {"seats", 8}]
[Section 5] checkAvailability() for Farmers Market: false
[Section 5] checkAvailability() for Jazz Night: true
[Section 4] Music registrations so far: 1
[Section 4] Music registrations so far: 2
[Section 4] filterEventsByCategory (Music): Live Jazz Night, Open Mic Music Evening
[Section 10] Event: Live Jazz Night [Music] at Riverside Cafe
[Section 10] Event: undefined [undefined] at undefined
[Section 10] Cloned list length: 6 | Original length: 6
[Section 3] Hiding event "Local Farmers Market" (past date or full).
[Section 3] Hiding event "Kids Story Time" (past date or full).
[Section 3] Hiding event "Local Farmers Market" (past date or full).
[Section 3] Hiding event "Kids Story Time" (past date or full).
[Section 1] Page fully loaded.
[Section 8] Category filter changed to: Music
[Section 3] Hiding event "Local Farmers Market" (past date or full).
[Section 3] Hiding event "Kids Story Time" (past date or full).
[Section 8] Category filter changed to: All
[Section 3] Hiding event "Local Farmers Market" (past date or full).
[Section 3] Hiding event "Kids Story Time" (past date or full).
[Section 8] Search box keydown, current value: Y
[Section 3] Hiding event "Local Farmers Market" (past date or full).
[Section 3] Hiding event "Kids Story Time" (past date or full).
[Section 8] Search box keydown, current value: Yo
[Section 3] Hiding event "Local Farmers Market" (past date or full).
[Section 3] Hiding event "Kids Story Time" (past date or full).
[Section 8] Search box keydown, current value: Yog
[Section 3] Hiding event "Local Farmers Market" (past date or full).
[Section 3] Hiding event "Kids Story Time" (past date or full).
[Section 8] Search box keydown, current value: Yoga
[Section 3] Hiding event "Local Farmers Market" (past date or full).
[Section 3] Hiding event "Kids Story Time" (past date or full).
[Section 8] Search box keydown, current value:
[Section 3] Hiding event "Local Farmers Market" (past date or full).
[Section 3] Hiding event "Kids Story Time" (past date or full).
[Section 3] Registered for Community Yoga Morning. Seats left: 14
[Section 9] async/await loaded 6 events from mock API.
[Section 9] fetch().then() loaded 6 events from mock API.
[Section 13] Validation step, errors found: {"name": "Please enter your full name.", "email": "Please enter a valid email
address."}
[Section 13] Validation step, errors found: {}
[Section 13] Submitting form with payload: {"name": "Steeve Rogers", "email": "steeve@example.com", "eventId": "3"}
[Section 12] Server response: {"status": "success", "message": "Registration confirmed for Steeve Rogers!"}

```

Screenshot: full console / debug log captured from the running application

14. jQuery and JS Frameworks

jQuery is used to simplify a DOM toggle: clicking the button fades an info panel in or out:

```

$(function () {
  $("#toggleInfoBtn").click(function () {
    if ($("#infoPanel").is(":visible")) {
      $("#infoPanel").fadeOut();
    } else {
      $("#infoPanel").fadeIn();
    }
  });
});

```

Local Community Event Portal

Upcoming Events

All Categories ▼

Community Yoga Morning

Fitness | Central Park
Date: 2026-07-20
Seats left: 14

[Register](#)

Live Jazz Night

Music | Riverside Cafe
Date: 2026-07-25
Seats left: 8

[Register](#)

Workshop on Baking

Workshop | Community Hall
Date: 2026-07-30
Seats left: 5

[Register](#)

Open Mic Music Evening

Music | Riverside Cafe
Date: 2026-08-02
Seats left: 12

[Register](#)

Load Events From Server (Async / Fetch API)

Loaded 6 events from server successfully.

Event Registration Form

Name

Email

Select Event

Community Yoga Morning ▼[Register](#)

Success: Registration confirmed for Steeve Rogers!

jQuery Demo

This portal currently uses plain JavaScript and jQuery. Moving to a framework like React or Vue would let us build the event cards as reusable components and automatically re-render the UI whenever the underlying event data changes, instead of manually updating the DOM.

Debug Log (Console Output)

```
[Section 2] Event: Live Jazz Night | Date: 2026-07-25 | Seats left: 8
[Section 2] Seat added back, now: 9
[Section 2] Seat taken by registration, now: 8
[Section 6] Total events after push(): 6
[Section 6] Music events: Live Jazz Night, Open Mic Music Evening
[Section 6] Display cards:
Card: Community Yoga Morning (Fitness)
Card: Local Farmers Market (Market)
Card: Live Jazz Night (Music)
Card: Kids Story Time (Kids)
Card: Workshop on Baking (Workshop)
Card: Open Mic Music Evening (Music)
[Section 5] Object.entries(event #3): [{"id",3}, {"name","Live Jazz Night"}, {"category","Music"}, {"location","Riverside Cafe"}, {"date","2026-07-25"}, {"seats",8}]
[Section 5] checkAvailability() for Farmers Market: false
[Section 5] checkAvailability() for Jazz Night: true
[Section 4] Music registrations so far: 1
```

Screenshot: info panel faded in after clicking the toggle button

One benefit of moving to a framework like React or Vue: the event cards could be written as reusable components that automatically re-render whenever the underlying event data changes, instead of manually creating and updating DOM elements by hand as done above.

End of Exercises